

Programming Assignment 6: Linked Lists

Due: Thursday, March 12th at 11pm CT

Learning Goals

- To implement a linked data structure
- To practice working with and managing references
- To understand and analyze the differences between linked lists and array-based lists

Introduction

As discussed in lecture, array-based lists store data in contiguous chunks of memory, which is great for quick access time but not ideal for certain insertions and deletions. Linked data structures provide an alternative to array-based data structures by allowing us to store data in non-contiguous chunks of memory. In order to extend your understanding of this concept, we want you to implement your very own doubly-linked list!

In this assignment, you will be using doubly-linked nodes, for which we provide a basic implementation, to implement a linked list class. A doubly-linked node has a reference to the next node in the list as well as a reference to the previous node in the list. Take a look at the nested `DoubleListNode` class in `LL314.java` to get a better understanding of this structure.

Assignment Guidelines and Specifications

This is an individual assignment, which means you cannot work with a partner on this assignment. All code submitted must be entirely your own.

For this assignment, there are some additional specifications:

- You may not use the Java `LinkedList` class or any other data structures from the Java standard library.
- You must implement the underlying storage container of your list as a doubly-linked linear structure using the provided `DoubleListNode` class.
- The data elements in the list may not be null. If null were allowed to be stored in the list, list implementations that rely on header nodes or references to the first and last nodes would not work.
- **You must state the Big O of every public method in LL314 in the corresponding method comment.**
- You must add conditional statements to check the preconditions for public methods.
- You may (and are encouraged to) create private helper methods to remove redundancy and provide structure to your solution.
- You may create and share tests with other students; however, the tests you submit must be your own. We encourage you to use Ed Discussion to share tests with each other.

Getting Started

Here are the necessary files for this assignment:

- [IList.java](#): This file contains the interface that your LL314 class will implement. Do not submit this file.
- [LL314.java](#): This file contains your implementation of a doubly-linked list as well as the provided implementation of the `DoubleListNode` class. You will submit this file.
- [LinkedListTester.java](#): This file contains the test harness for LL314. Ensure your implementation passes the given tests before removing them and creating your own tests (two for each public method in LL314). You will submit this file.
- [Stopwatch.java](#): This file contains the `Stopwatch` class which is used to measure the time elapsed in the comparison experiments. Do not submit this file.

Implementing a doubly-linked list is by no means a trivial pursuit. There are a host of common issues that we've seen students run into. Read through these tips before you start coding and refer back to them regularly:

1. **Implementation:** You may choose to implement the doubly-linked list in many ways. Some common approaches include creating a null-terminated list (which often has references to the first and last nodes) and implementing a circular list (which often has a reference to a header node that doesn't contain any data). There are advantages and disadvantages to each approach, but no one approach is generally better than any other approach. The one you choose is up to you. We highly recommend you watch the posted video on circular, doubly-linked lists (which can be found on Panopto Video on Canvas) and refer to the slides (which can be found [here](#)) before making your decision.
2. **Traversal:** Consider the benefits of using a doubly-linked list over a singly-linked list. When searching for a particular index in your list, is there a way you can determine the quickest path to the element at that index? For example, what if you wish to retrieve the second-to-last element in the list? Would it make sense to begin searching for this element at the beginning of the list? Make sure you use the features of a doubly-linked list to your advantage in order to increase the efficiency of your methods.
3. **Redundancy:** Some of the methods can be easily implemented using other methods from the class. For example, the `addLast` method is very similar to the `add` method (in fact, the `add` method and the `insert` method also have very similar functions). See where you can reduce redundancy by implementing some methods through passing in specific arguments to other methods. However, be careful not to harm the Big O of methods when reducing redundancy. Prioritize efficiency over redundancy reduction.
4. **Testing:** Avoid implementing multiple methods at a time before testing. Instead, implement one method, then test it, and follow that pattern as much as you can to ensure your methods are behaving as expected. When debugging, draw diagrams of the expected behavior versus the actual behavior to diagnose logic errors. Note that some of the given test cases call and rely on multiple different methods. For example, Test 10, which tests `remove`, also makes calls to `add`. If this test is failing, you should check that all of the methods called, both `remove` *and* `add`, are working correctly.

LL314

Your LL314 class should implement the given `IList` interface, which means that you will have to implement every method that is declared in `IList` within the LL314 class. The following methods are inherited from the `IList` interface:

1. Implement the following method:

```
public void add(E item)
```

This method adds the data `item` to the end of the list. This method has the precondition that `item` cannot be equal to null.

2. Implement the following method:

```
public void insert(int pos, E item)
```

This method inserts the data `item` at index `pos` within the list. This method has the preconditions that `pos` must be from 0 to the size of the list (inclusive) and that `item` cannot be equal to null.

3. Implement the following method:

```
public E set(int pos, E item)
```

This method changes the data at index `pos` to `item` and returns the old data at index `pos`. This method has the preconditions that `pos` must be between 0 (inclusive) and the size of the list (exclusive) and that `item` cannot be equal to null.

4. Implement the following method:

```
public E get(int pos)
```

This method returns the data at index `pos`. This method has the precondition that `pos` must be between 0 (inclusive) and the size of the list (exclusive).

5. Implement the following method:

```
public E remove(int pos)
```

This method removes and returns the element at index `pos` from the list. This method has the precondition that `pos` must be between 0 (inclusive) and the size of the list (exclusive).

6. Implement the following method:

```
public boolean remove(E obj)
```

This method removes the first instance of the data `obj` in the list. If `obj` is in the list and is removed, the method returns `true`. If `obj` is not in the list, the method returns `false`. This method has the precondition that `obj` cannot be equal to `null`.

7. Implement the following method:

```
public IList<E> getSubList(int start, int stop)
```

This method returns a sublist of the elements in the calling list from the index `start` (inclusive) to the index `stop` (exclusive). The calling list should not be altered. This method has the preconditions that `start` must be between 0 (inclusive) and the size of the list (inclusive) and that `stop` must be between `start` (inclusive) and the size of the list (inclusive).

8. Implement the following method:

```
public int size()
```

This method returns the number of elements in the calling list.

9. Implement the following method:

```
public int indexOf(E item)
```

This method searches for `item` within the calling list and returns the index of the first element equal to `item`. If `item` is not present within the list, this method returns `-1`. This method has the precondition that `item` cannot be equal to `null`.

10. Implement the following method:

```
public int indexOf(E item, int pos)
```

This method searches for `item` within the calling list starting at index `pos` and returns the index of the first element starting at `pos` equal to `item`. If `item` is not present within the list starting at index `pos`, this method returns `-1`. This method has the preconditions that `pos` must be between 0 (inclusive) and the size of the list (exclusive) and that `item` cannot be equal to `null`.

11. Implement the following method:

```
public void makeEmpty()
```

This method returns the list to an empty state where there are no elements in the list.

12. Implement the following method:

```
public Iterator<E> iterator()
```

This method returns an iterator for the calling list.

Note that just because you are creating an iterator for the list does **not** mean that you are required to use an iterator in your implementation of your other methods.

The methods within the LL314 iterator should all be $O(1)$. You will need to implement the `hasNext` and `next` methods within the iterator, as well as replace the default `remove` method with an implementation that actually removes the node containing the last value returned by `next` from the calling list.

We highly recommend you watch the posted video on iterators for singly-linked lists (which can be found on Panopto Video on Canvas) and refer to the slides (which can be found [here](#)) before implementing the iterator for the doubly-linked list.

13. Implement the following method:

```
public void removeRange(int start, int stop)
```

This method removes a range of elements from index `start` (inclusive) to index `stop` (exclusive). This method has the preconditions that `start` must be between 0 (inclusive) and the size of the list (inclusive) and that `stop` must be between `start` (inclusive) and the size of the list (inclusive).

14. Implement the following method:

```
public String toString()
```

This method returns a `String` representation of the list in the form $[e_0, e_1, \dots, e_n]$. The `String` representation includes square brackets around the list, with the elements in order based on their position in the list, separated by a comma and a space.

15. Implement the following method:

```
public boolean equals(Object other)
```

This method returns a boolean indicating whether the calling list is equal to `other`. Two `ILists` are equal if they contain the same elements in the same order. This means that an `LL314` can be equal to an `IList` that isn't an `LL314` so long as the aforementioned condition is satisfied.

Note that two empty lists are equal to each other regardless of the type of elements that they store (as is true in the Java Standard Library). When determining whether one

`IList` is equal to another `IList` within your `equals` method, it is unsafe to assume that the first `IList` contains the same type of generic object as the second `IList`. Thus, you should avoid casting to an `IList<E>` when determining equality. Revisit section 7.2 of the style guide for further clarification.

The following methods are specific to the `LL314` class:

16. Implement the relevant `LL314` constructor(s) based on your implementation.

17. Implement the following method:

```
public void addFirst(E item)
```

This method adds the data `item` to the front of the linked list. This method has the precondition that `item` cannot be equal to null.

18. Implement the following method:

```
public void addLast(E item)
```

This method adds the data `item` to the end of the linked list. This method has the precondition that `item` cannot be equal to null.

19. Implement the following method:

```
public E removeFirst()
```

This method removes and returns the first element of the linked list. This method has the precondition that `size` must be greater than 0.

20. Implement the following method:

```
public E removeLast()
```

This method removes and returns the last element of the linked list. This method has the precondition that `size` must be greater than 0.

Testing

You are required to write and submit your own test cases in `LinkedListTester.java`. You are expected to write:

- 2 new test cases for `add`
- 2 new test cases for `insert`
- 2 new test cases for `set`
- 2 new test cases for `get`
- 2 new test cases for `remove(pos)`
- 2 new test cases for `remove(obj)`
- 2 new test cases for `getSubList`
- 2 new test cases for `size`
- 2 new test cases for `indexOf(item)`
- 2 new test cases for `indexOf(item, pos)`
- 2 new test cases for `makeEmpty`
- 2 new test cases for `iterator.hasNext`
- 2 new test cases for `iterator.next`
- 2 new test cases for `iterator.remove`
- 2 new test cases for `removeRange`
- 2 new test cases for `toString`
- 2 new test cases for `equals`
- 2 new test cases for `addFirst`
- 2 new test cases for `addLast`
- 2 new test cases for `removeFirst`
- 2 new test cases for `removeLast`

In previous assignment guides, we shared a few notes with you about writing test cases. Those notes are copied here for your convenience:

- You may copy the structure of the provided tests.
- Any test cases that you write should not violate the preconditions of the method.
- You must group the test cases by method and comment which method the test case is for.
- You are not required to write test cases for any private helper methods or constructors.
- **Delete the test cases that we have provided for you.**

Experiments

As mentioned in lecture, there are some interesting differences in behavior between linked lists and array-based lists. We have provided a means of testing these differences within the `LinkedListTester` class.

When your `LL314` class is finished, run the `comparison` method from the `LinkedListTester` class to perform a number of tests on your `LL314` class and the Java `ArrayList` class. To run the method, simply uncomment the line at the end of the main

method in `LinkedListTester` that calls the comparison method. You are free to change the initial value of `N` for the tests in the comparison method.

In a comment at the top of your `LinkedListTester` class, indicate which operations are faster when using your `LL314`, which are faster when using the `Java ArrayList` class, and which are about the same. Make sure to include the timing data you used to come to these conclusions within the comment. Note that the experiments use different starting values of `N`.

For each method tested in this experiment, state what you think the Big O is (for both the `LL314` class and the `Java ArrayList` class) based on the timing data. Include your reasoning in the comment as well.

Submission Checklist

Before you submit, run through our submission checklist to make sure you didn't forget anything! Did you remember to:

1. Implement all the required methods?
2. Ensure each method in your `LL314` class states the Big O in the method comment?
3. Ensure your program passes the given tests in `LinkedListTester.java`?
4. Create and include your own test cases?
5. Place the results of your experiments and conclusions in a comment at the top of `LinkedListTester.java`?
6. Fill in the header comments for `LL314.java` and `LinkedListTester.java`?
7. Check the preconditions in public methods and throw exceptions as necessary?
8. Follow the [program hygiene guide](#)?
9. Follow the assignment specifications from page 3 of the [Assignment handout](#)?

If you have done all these things, then you're ready to submit `LinkedListTester.java` and `LL314.java` to Gradescope!

Thank you to Professor Mike Scott for creating the previous version of this assignment!